Introducción a JavaScript(.JS): es un lenguaje de programación que se ejecuta en el navegador y permite manipular el contenido **HTML**, manejar eventos, y modificar estilos, entre otras cosas.

Hay dos formas principales de escribir **JavaScript:**

- JS tradicional (antes del estándar ES6)
- JS moderno (ES6+) que incluye nuevas características como let, const, funciones de flecha, clases, entre otras.

Declarar variables:

Si bien se puede declarar con `var`, no es muy conveniente porque estas al ser globales pueden ser utilizadas sin haber declarado la variable propiamente dicha primero, por lo que genera inconsistencias en el código.

`const:` variable constante que no puede ser reasignada.

`let:` permite declarar variables con alcance de bloque, **(moderno y recomendado)** es decir, solamente se puede usar dentro de un condicional o un bloque de código.

```
AprendeJS.js
1  {
2
3   let x = 10; // esto es un bloque de codigo
4
5  }
6
7  console.log(x); // aqui no vamos a poder usarla
8
```

```
AprendeJS.js
1
2  // Ejemplo de variable declarada con 'var'
3  var nombre = "bob";
4
5  // Variable usando 'let' y  'const'
6
7  let nombre = 'Juan';  // Variable que puede ser reasignada
8  const edad = 30;      // Variable constante, no se puede reasignar
```

Tipos de datos:

Primitivos: string, number, boolean, undefined, null, symbol, bigint.
No Primitivos: Objetos, Arrays, Funciones.

AprendeJS.js

```javascript
let cadena = "Hola Mundo"; // string (cadena de texto)
let numero = 42;           // Numerico
let esActivo = true;       // Boolean
let objeto = { nombre: 'Juan', edad: 30 }; // Objeto
```

Estructuras de control:

AprendeJS.js

```javascript
//ESTRUCTURA IF-ELSE
let edad = 18;

if (edad >= 18) {
  console.log('Eres mayor de edad');
} else {
  console.log('Eres menor de edad');
}

// SWITCH (Util para multiples condiciones)
let dia = 2;
switch (dia) {
  case 1:
    console.log('Lunes');
    break;
  case 2:
    console.log('Martes');
    break;
  default:
    console.log('Otro día');
}

// Bucles (FOR iteracion tradicional)
for (let i = 0; i < 5; i++) {
  console.log(i);
}
//WHILE iteracion basada en condicion
let i = 0;
while (i < 5) {
  console.log(i);
  i++;
}
```

```
/* FOReach solo para arreglos */
let frutas = ['manzana', 'naranja', 'plátano'];
frutas.forEach(function(fruta) {
  console.log(fruta);
});
```

Arreglos (arrays)

`.find():` Encuentra el primer elemento que cumpla con una condición.
`.map():` Crea un nuevo arreglo transformando cada elemento.
`.append():` Agrega un elemento al final de un contenedor en el **DOM.**
`.push():` Agrega un elemento al final de un arreglo.
`.filter():` Crea un nuevo arreglo con los elementos que pasen una condición.
`.reduce():` Reduce todos los elementos de un arreglo a un único valor.
`.forEach():` Ejecuta una acción para cada elemento del arreglo.

AprendeJS.js

```
/*
Arreglos Tradicionales:
Se declaran con [] y pueden almacenar cualquier tipo de datos.
*/
let frutas = ['manzana', 'naranja', 'plátano'];
```

Métodos comunes con **Arreglos:**
Acceso a elementos: **`frutas[0]`**
Añadir al final: **`frutas.push('uva')`**
Eliminar del final: **`frutas.pop()`**
Eliminar del inicio: **`frutas.shift()`**
Añadir al inicio: **`frutas.unshift('fresa')`**
Iterar: **`forEach()`**

Objetos **(Arreglos Asociativos)** en **" .JS "** es una colección de pares **clave-valor**. Se usa para representar estructuras más complejas.

```
AprendeJS.js

let persona = {
  nombre: 'bob',
  edad: 25,
  saludar: function() {
    console.log('Hola, soy ' + this.nombre);
  }
};
//Acceder a propiedades de objetos:
console.log(persona.nombre);  // "bob"
persona.saludar();            // "Hola, soy bob"
```

Funciones: permiten encapsular lógica que puedes reutilizar.

```
AprendeJS.js

Sintaxis Tradicional:

function saludar() {
  console.log('Hola');
}
saludar();

// ========================== //

Funciones de Flecha (ES6+):

const saludar = () => {
  console.log('Hola');
};
saludar();
```

Clases y objetos

```
AprendeJS.js

Clases y Objetos (ES6+)
Con ES6, JavaScript introduce una sintaxis más clara para definir clases.

class Persona {
  constructor(nombre, edad) {
    this.nombre = nombre;
    this.edad = edad;
  }

  saludar() {
    console.log('Hola, soy ' + this.nombre);
  }
}

let bob = new Persona('bob', 25);
bob.saludar();  // "Hola, soy bob"
```

Encapsulamiento **(Manejo de propiedades privadas y públicas)**
En **ES6+**, puedes crear propiedades y métodos privados usando el prefijo #.

```
AprendeJS.js

class Persona {
  #nombre; // propiedad privada

  constructor(nombre, edad) {
    this.#nombre = nombre;
    this.edad = edad;
  }

  saludar() {
    console.log('Hola, soy ' + this.#nombre);
  }
}

let juan = new Persona('Juan', 30);
juan.saludar();  // "Hola, soy Juan"
// console.log(juan.#nombre);  // Error: #nombre es privado
```

Manejo de **DOM (Document Object Model)**

Se refiere a la forma en que los navegadores web representan y permiten la interacción con el contenido de una página web, como si fuera un árbol de objetos. El **DOM** es una estructura que organiza el contenido de una página **HTML (o XML)** de manera jerárquica, lo que

permite que los programadores puedan modificar, agregar o eliminar elementos de la página mediante programación, generalmente utilizando **JavaScript.**

Con el manejo del **DOM** puedes:

- Cambiar el texto de un párrafo.
- Modificar el estilo de un elemento **(como el color de un botón)**.
- Agregar nuevos elementos a la página, como imágenes o enlaces.
- Eliminar elementos existentes, como una lista de elementos.

AprendeJS.js

```javascript
/* Acceder a elementos del DOM

getElementById: Obtiene un elemento por su ID.
*/
let elemento = document.getElementById('miElemento');

//getElementsByClassName: Obtiene elementos por su clase.
let elementos = document.getElementsByClassName('miClase');
//querySelector: Selecciona el primer elemento que coincide con el selector CSS.
let primerElemento = document.querySelector('.miClase');
//querySelectorAll: Selecciona todos los elementos que coinciden con el selector CSS.
let todosElementos = document.querySelectorAll('.miClase');

/*
Manipular contenido y atributos:

innerHTML: Cambiar el contenido HTML de un elemento.
 */
let elemento = document.getElementById('miElemento');
elemento.innerHTML = 'Nuevo contenido';

/* textContent: Cambiar solo el texto de un elemento.*/
let elemento = document.getElementById('miElemento');
elemento.textContent = 'Nuevo texto';

//setAttribute(): Cambiar un atributo de un elemento.
let imagen = document.querySelector('img');
imagen.setAttribute('src', 'nuevaImagen.jpg');

/*Modificar Estilos:
style: Cambiar el estilo de un elemento.*/
let elemento = document.getElementById('miElemento');
elemento.style.backgroundColor = 'blue';
elemento.style.fontSize = '20px';
```

AprendeJS.js

```javascript
//Crear y eliminar elementos:
//createElement(): Crear un nuevo elemento.
let nuevoDiv = document.createElement('div');
nuevoDiv.textContent = 'Hola Mundo';
document.body.appendChild(nuevoDiv);  // Añadir al final del body

//removeChild(): Eliminar un hijo del DOM.
let elemento = document.getElementById('miElemento');
document.body.removeChild(elemento);

//Manejo de Eventos:
//addEventListener(): Añadir un evento a un elemento.
let boton = document.getElementById('miBoton');
boton.addEventListener('click', function() {
  alert('¡Botón clickeado!');
});
//removeEventListener(): Eliminar un evento de un elemento.
boton.removeEventListener('click', miFuncion);
```

Ejemplo de manejo de DOM:

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejemplo de JS</title>
</head>
<body>
  <div id="miElemento">¡Haz clic en el botón!</div>
  <button id="miBoton">Clickeame</button>

  <script>
    let boton = document.getElementById('miBoton');
    boton.addEventListener('click', function() {
      let elemento = document.getElementById('miElemento');
      elemento.textContent = '¡El botón fue clickeado!';
      elemento.style.color = 'red';
    });
  </script>
</body>
</html>
```

tip:

AprendeJS.js

```javascript
console.log() //una de las formas
//más comunes de imprimir información en la consola, JavaScript proporciona
//varias otras formas para interactuar con el
//entorno de desarrollo y el navegador.

console.error("¡Este es un mensaje de error!");
//Utilizado para imprimir mensajes de error.

console.warn() //mensajes de advertencia

console.info() //mensajes de informacion

/*
console.table(); Permite imprimir un objeto o un array en formato tabular,
lo cual es muy útil cuando se trabaja con colecciones de datos.
Esto hace que la información sea más fácil de leer.
*/
let personas = [
  { nombre: "Juan", edad: 30 },
  { nombre: "Ana", edad: 25 },
  { nombre: "Pedro", edad: 40 }
];

console.table(personas);
```

AprendeJS.js

```javascript
alert()//cuadro de diálogo con un mensaje en el navegador.

alert("¡Hola, este es un mensaje de alerta!");

prompt()//cuadro de diálogo que permite al usuario ingresar un valor.

let nombre = prompt("¿Cuál es tu nombre?");
console.log("Hola, " + nombre);

confirm()
//cuadro de diálogo con un mensaje y dos botones
//("Aceptar" y "Cancelar"). Devuelve true si el usuario hace clic en "Aceptar",
// false si hace clic en "Cancelar". Es útil para obtener confirmación antes de realizar una acción.
let respuesta = confirm("¿Estás seguro de que deseas continuar?");
if (respuesta) {
  console.log("El usuario ha aceptado.");
} else {
  console.log("El usuario ha cancelado.");
}

//Document.write() permite escribir directamente en el HTML
document.write("Este es un mensaje escrito directamente en la página.");
//no se recomienda para el desarrollo moderno, ya que puede sobrescribir
//el contenido de la página si se usa después de que el contenido ya haya sido cargado.

console.assert()
//Imprime un mensaje si el primer parametro que se manda es falso. (util para depurar)
let x = 5;
console.assert(x > 10, "El valor de x es menor que 10");

console.count()
//imprime la cantidad de veces que ha sido llamada una etiqueta
```

```
AprendeJS.js

console.trace()
Imprime la traza de la pila, lo cual te ayuda a seguir el camino de ejecución del código
y localizar en qué función o parte del código se generó un error o se llamó a una función.


function funcionA() {
  funcionB();
}

function funcionB() {
  console.trace("Rastro de la llamada");
}

funcionA();
```

- **console.log():** Muestra mensajes generales en la consola.
- **console.error():** Muestra mensajes de error.
- **console.warn():** Muestra advertencias.
- **console.info():** Muestra mensajes informativos.
- **console.table():** Muestra datos en formato tabular.
- **alert():** Muestra un cuadro de alerta **(interrumpe el flujo)**.
- **prompt():** Solicita datos al usuario mediante un cuadro de entrada.
- **confirm():** Solicita confirmación del usuario **(sí/no)**.
- **document.write():** Escribe directamente en el documento **HTML**.
- Manipulación del **DOM**: Modifica el contenido **HTML** o los estilos directamente en la página.

Operadores Aritméticos

- **Adición (+):** Suma dos valores.
- **Sustracción (-):** Resta un valor de otro.
- **Multiplicación (*):** Multiplica dos valores.
- **División (/):** Divide un valor entre otro.
- **Módulo (%):** Devuelve el residuo de la división entre dos números.
- **Exponentiation (**):** Eleva un número a la potencia de otro.

Operadores de Comparación

- **Mayor que (>)**
- **Menor que (<)**
- **Mayor o igual que (>=)**
- **Menor o igual que (<=)**
- **Igual a (==)**
- **Identidad estricta (===)**
- **Desigual a (!=)**
- **Desigual a estricta (!==)**

Operadores Lógicos (para combinar **condiciones booleanas**)

AND (&&): Devuelve true **sólo** si ambas condiciones son verdaderas.
OR (||): Devuelve true si al menos una de las condiciones es verdadera.
NOT (!): Invierte el valor de una condición **(de true a false y viceversa).**

Concatenación de Cadenas

- **Concatenación con +**

console.log(saludo + " " + nombre**);**

Concatenación con **template literals** (plantillas literales): utilizando las comillas invertidas `(backticks)` y `${}` para insertar valores directamente dentro de la cadena.

console.log(``${saludo} ${nombre}``); /* Hola bob */

JavaScript: Paradigma Basado en Prototipos

JavaScript no utiliza clases en su núcleo, sino que tiene un **modelo basado en prototipos**. Esto significa que **cada objeto** en JavaScript tiene un **objeto prototipo asociado**, y ese prototipo también puede tener su propio prototipo, formando una cadena de prototipos **(también conocida como prototype chain)**. Esta es la **base de la herencia** en JavaScript

¿Cómo funciona esto?
En lugar de crear una clase y luego instanciar objetos de esa clase **(como en lenguajes orientados a objetos tradicionales, como Java o C++)**, en **JavaScript puedes crear objetos directamente y asignarles un prototipo** que **comparta métodos y propiedades comunes.**

```javascript
// Crear un objeto "animal" con un prototipo
let animal = {
  comer: function() {
    console.log("El animal está comiendo.");
  }
};

// Crear un objeto "perro" que hereda de "animal"
let perro = Object.create(animal);
perro.ladrar = function() {
  console.log("¡Guau!");
};

perro.comer();  // "El animal está comiendo."
perro.ladrar(); // "¡Guau!"

//animal es un objeto que tiene un método comer().

//Usamos Object.create(animal) para crear un nuevo objeto (perro)
//que hereda el prototipo de animal.

//El objeto perro tiene su propio método ladrar(),
//pero también puede acceder al método comer() que heredó de animal.
```

super() Es una función especial que se usa en el contexto de las clases para hacer referencia al constructor y a los métodos de la **clase padre (superclase)**. Se utiliza principalmente en dos situaciones:

- Llamar al constructor de la clase padre (cuando estás usando herencia).
- Llamar a métodos de la clase padre desde un método de la clase hija.

```javascript
// Clase Padre
class Animal {
  constructor(nombre) {
    this.nombre = nombre;
    console.log("Animal creado: " + nombre);
  }

  comer() {
    console.log(this.nombre + " está comiendo.");
  }
}

// Clase Hija (Perro) que hereda de Animal
class Perro extends Animal {
  constructor(nombre, raza) {
    super(nombre); // Llama al constructor de la clase padre (Animal)
    this.raza = raza;
    console.log("Perro creado: " + nombre + " de raza " + raza);
  }

  ladrar() {
    console.log("¡Guau!");
  }
}

const miPerro = new Perro("Fido", "Labrador");
```